

IBKR_Connection.py

```

1  import datetime, time
2  import threading
3  from ibapi.client import EClient
4  from ibapi.wrapper import EWrapper
5  from ibapi.connection import *
6
7  class ConnectionManager(EWrapper,EClient):
8      def __init__(self):
9          EClient.__init__(self,self)
10         EWrapper.__init__(self)
11         self._lock=threading.Lock()
12         self.reqID_counter:int=1
13         self.next_valid_order_ID=None
14         self.connected=False
15         self.callbacks:dict={}
16         self.data_ready=threading.Event()
17
18     def next_reqID(self):
19         with self._lock:
20             self.reqID_counter+=1
21             return self.reqID_counter
22
23     def next_orderID(self):
24         if self.next_valid_order_ID is None:
25             raise RuntimeError("nextValidId is not received yet. Please try after some time!")
26         oid=self.next_valid_order_ID
27         self.next_valid_order_ID+=1
28         return oid
29
30     def nextValidId(self, orderId):    # callback from IBKR
31         """IBKR calls this when its connected"""
32         self.next_valid_order_ID=orderId
33         self.data_ready.set()        # signal that connection is ready
34         print(f"Next valid orderId is received:{orderId}")
35
36     def error(self,reqId:int,errorCode:int,errorString:str,advancedOrderRejectJson:str=""):
37         print(f"[IBKR ERROR] reqId={reqId} code={errorCode} msg={errorString}")
38         if advancedOrderRejectJson:
39             print(f"Advanced reject:{advancedOrderRejectJson}")
40
41     def
42 connect(self,host:str="127.0.0.1",port:int=7497,clientId:int=1,max_attempts:int=11,delay_seconds:int=5):
43     # check the connection is already established or not
44     if self.connected and getattr(self,"thread",None) and self.thread.is_alive():
45         raise RuntimeError("Already connected")
46     # try and error of the connectivity
47     attempt=1
48     last_error=None
49     while attempt<max_attempts:
50         try:
51             print(f"[Attempt {attempt}/{max_attempts}] Connecting to {host}:{port}")
52             self.data_ready.clear()    # clear previous state
53             EClient.connect(self=self,host=host,port=port,clientId=clientId)
54             self.thread=threading.Thread(target=self.run,daemon=True)
55             self.thread.start()
56             self.connected=True
57             print(f"IBKR connection established on attempt {attempt}")
58             return

```

```
58         except Exception as e:
59             last_error=e
60             self.connected=False
61             if attempt<max_attempts:
62                 print(f"Attempt {attempt} failed: {e}")
63                 time.sleep(delay_seconds)
64                 attempt+=1
65             else:
66                 break
67         raise RuntimeError(f"IBKR connection fail after {max_attempts} attempts|{last_error}")
68
69     def reconnect(self,host:str="127.0.0.1",port:int=7497,clientId:int=1):
70         self.disconnect()
71         self.connect(host=host,port=port,clientId=clientId)
72
73     def disconnect(self):
74         if not self.connected and not getattr(self,"thread",None):
75             raise RuntimeError("Already Disconnected")
76         self.connected=False
77         EClient.disconnect(self)
78         self.data_ready.clear()
79         self.next_valid_order_ID=None
80         if hasattr(self,"thread") and self.thread.is_alive():
81             self.thread.join(timeout=2)
82
83     def wait_for_ready(self,timeout:int=10):
84         """Block until nextvalidId is received, meaning the API is ready"""
85         if not self.data_ready.wait(timeout=timeout):
86             raise TimeoutError("IBKR connection did not became ready in time")
87
88     def connectionClosed(self):      # IBKR wrapper callback when the connection is closed
89         self.connected=False
90         print("IBKR connection is closed")
91
92     def connectAck(self):             # optional IB Wrapper callback in newer API versions
93         print("IBKR connection acknowledged")
94         self.data_ready.set()
95
96     def isConnected(self):           # check the current connection status
97         return self.connected and EClient.isConnected(self)
```